S.No: 1	Exp. Name: ***Registration Form using Table***	**Date: 2026-08-12**

ID: A24126552275

ANIL NEERUKONDA INSTITUTE OF TECHNOLOGY & SCIENCES

Aim:
Build a registration form laid out inside an HTML table.

Form Fields

- **Name**: A text input with `name="name"`.
- **Password**: A password input with `name="password"`.
- **E-mail**: An email input with `name="email"`.
- **Phone Number**: A telephone input with `type="tel"` and `name="phone"`.
- **Sex**: Two radio buttons for "Male" and "Female", both with `name="sex"`.
- **Date of Birth**: Three dropdowns with `name="day"`, `name="month"`, and `name="year"`.
- **Languages Known**: Four checkboxes for English, Telugu, Hindi, and Tamil, each with a matching lowercase `name` attribute.
- **Address**: A textarea with `name="address"`.

Table Layout

- Place each label in one column and its input in the adjacent column.
- Set the table border to 1.

Buttons

- A <button> element with `type="submit"` and the text "Submit".
- A <button> element with `type="reset"` and the text "Reset".

The form should look like this.

Registration Form

Name:	
Password:	
E-mail:	
Phone Number:	
Sex:	○ Male ○ Female
Date of Birth:	Day Month Year
Languages Known:	☐ English ☐ Telugu ☐ Hindi ☐ Tamil
Address:	
Submit Reset	

Source Code:

index.html

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-
scale=1.0">
    <title>Registration Form</title>
</head>
<body>
<h2>Registration Form</h2>
<!--Write your code here...-->
<form>
        <table border="1">
                <tr>
                        <td><label for="name">Name:</label></td>
                        <td><input type="text" id="name"
name="name"></td>
                </tr>
                <tr>
                        <td><label for="password">Password:
</label></td>
                        <td><input type="password" id="password"
name="password"></td>
                </tr>
                <tr>
                        <td><label for="email">E-mail:</label>
</td>
                        <td><input type="email" id="email"
name="email"></td>
                </tr>
                <tr>
                        <td><label for="phone">Phone Number:
</label></td>
                        <td><input type="tel" id="phone"
name="phone"></td>
                </tr>
                <tr>
                        <td><label>Sex:</label></td>
                        <td>
                                <input type="radio" id="male"
name="sex">
                                <label for="male">Male</label>
                                <input type="radio" id="female"
name="sex">
```

```
                                <label
for="female">Female</label>
                        </td>
                </tr>
                <tr>
                        <td><label>Date of Birth</label>
                        <td>
                                <select name="day" id="day">
                                        <option
value="">Day</option>
                                        <option
value="">01</option>
                                </select>
                                <select name="month" id="month">
                                        <option
value="">Month</option>
                                        <option
value="">01</option>
                                </select>
                                <select name="year" id="year">
                                        <option
value="">Year</option>
                                        <option
value="">2000</option>
                                </select>
                        </td>
                </tr>
                <tr>
                        <td><label>Languages Known:</label></td>
                        <td>
                                <input type="checkbox"
id="english" name="english">
                                <label for="english">English<br>
</label>
                                <input type="checkbox"
id="telugu" name="telugu">
                                <label for="telugu">Telugu<br>
</label>
                                <input type="checkbox" id="hindi"
name="hindi">
                                <label for="hindi">Hindi<br>
</label>
                                <input type="checkbox" id="tamil"
name="tamil">
                                <label for="tamil">Tamil</label>
                        </td>
```

```
                </tr>
                <tr>
                        <td><label for="address">Address:</label>
</td>
                        <td>
                                <textarea id="address"
name="address" rows="4" cols="20"></textarea>
                        </td>
                </tr>
                <tr>
                        <td colspan="2">
                                <button type="submit"
value="Submit">Submit</button>
                                <button type="reset"
value="Reset">Reset</button>
                        </td>
                </tr>
        </table>
</form>
</body>
</html>
```

S.No: 2	Exp. Name: ***Ravi's Portfolio Page***	**Date: 2026-08-28**

Aim:

Ravi is designing his personal portfolio webpage. He wants each section to look visually distinct. Style the text in each section as described below.

- In the `<header>`, the `<h1>` should have `font-family: Verdana`, `color: royalblue`, `text-shadow: 3px 3px 8px gray`, and `text-align: center`.
- In the `<main>`, the `<h2>` should have `text-transform: uppercase`, `color: darkred`, and `letter-spacing: 2px`.
- In the `<main>`, the `<p>` should have `font-size: 16px`, `color: #333`, and `line-height: 1.7`.
- In the `<footer>`, the `<p>` should have `text-align: center`, `font-style: italic`, and `color: darkgreen`.

Source Code:

ID: A24126552275

index.html

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8" />
  <meta name="viewport" content="width=device-width, initial-
scale=1.0"/>
  <title>Ravi’s Portfolio Page</title>
  <link rel="stylesheet" href="styles.css" />
</head>
<body>
  <header>
    <h1>Welcome to My Portfolio</h1>
  </header>

  <main>
    <h2>about me</h2>
    <p>Hello! I'm Ravi, a passionate front-end developer who
loves designing simple and elegant web experiences for users.</p>
  </main>

  <footer>
    <p>© 2025 Ravi Sharma | Designed with creativity and code.
</p>
  </footer>
</body>
</html>
```

styles.css

```
/*Write your code here*/
header h1 {
        font-family: verdana;
        color: royalblue;
        text-shadow: 3px 3px 8px gray;
        text-align: center;
}

main h2{
        text-transform: uppercase;
        color: darkred;
        letter-spacing: 2px;
}

main p{
        font-size: 16px;
        color: #333;
        line-height: 1.7;
}

footer p{
        text-align: center;
        font-style: italic;
        color: darkgreen;
}
```

S.No: 3	Exp. Name: ***Simple Calculator***	**Date: 2026-08-28**

Aim:

You are provided with an HTML file that defines the structure of a basic calculator. The calculator supports the following arithmetic operations:

- Addition (+)
- Subtraction (-)
- Multiplication (*)
- Division (/)

Your task is to complete the JavaScript file (script.js) so that the calculator functions correctly.

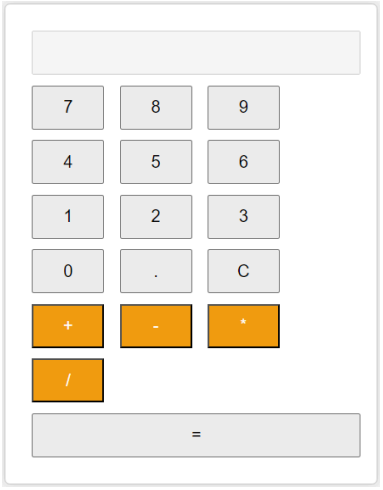
Provided Files

HTML File (index.html)

The HTML file already includes:

- A display area to show the current input or result.
- Buttons for digits (0–9).
- A decimal point button (.).
- Operation buttons (+, –, *, /).
- An equals button (=).
- A clear button (C).

You do not need to modify the HTML file.



Complete the JavaScript Functionality:

You must complete the logic in **script.js** by implementing the following functions correctly:

1. appendNumber(number)

This function is called when a number button is clicked.
It should:

- Allow multi-digit number input.
- Append the clicked digit to the current input.
- Handle decimal input properly (prevent multiple decimals in a single number).

2. setOperation(operation)

This function is called when an arithmetic operator (+, −, *, /) is clicked.
It should:

- Store the current input as the previous operand.
- Store the selected operation.
- Prepare the calculator to receive the next number.
- Prevent setting an operation if there is no current input.

3. calculate()

This function is called when the equals (=) button is clicked.
It should:

- Perform the selected arithmetic operation using the stored previous value and current value.
- Display the result.
- Handle division by zero by displaying:

`Cannot divide by zero`

If either the previous value or current value is empty, display:

`Error`

4. clearDisplay()

This function is called when the clear (C) button is clicked.
It should:

- Reset the current input.
- Reset the previous input.
- Reset the selected operation.
- Clear the display.

Source Code:

index.html

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-
scale=1.0">
    <title>Simple Calculator</title>
    <style>
        body {
            font-family: Arial, sans-serif;
            display: flex;
            justify-content: center;
            align-items: center;
            height: 100vh;
            background-color: #f4f4f4;
        }
        .calculator {
            border: 1px solid #ccc;
            padding: 20px;
            border-radius: 5px;
            background-color: #fff;
            box-shadow: 0 0 10px rgba(0,0,0,0.1);
        }
        .calculator input, .calculator button {
            margin: 5px;
            padding: 10px;
            font-size: 16px;
        }
        .calculator input {
            width: 100%;
            box-sizing: border-box;
        }
        .calculator button {
            width: 22%;
        }
        .calculator button.operator {
            background-color: #f39c12;
            color: #fff;
        }
    </style>
</head>
<body>
    <div class="calculator">
        <input type="text" id="display" disabled>
```

```
        <br>
        <button onclick="appendNumber('7')">7</button>
        <button onclick="appendNumber('8')">8</button>
        <button onclick="appendNumber('9')">9</button>
        <br>
        <button onclick="appendNumber('4')">4</button>
        <button onclick="appendNumber('5')">5</button>
        <button onclick="appendNumber('6')">6</button>
        <br>
        <button onclick="appendNumber('1')">1</button>
        <button onclick="appendNumber('2')">2</button>
        <button onclick="appendNumber('3')">3</button>
        <br>
        <button onclick="appendNumber('0')">0</button>
        <button onclick="appendNumber('.')">.</button>
        <button onclick="clearDisplay()">C</button>
        <button class="operator" onclick="setOperation('+')">+
</button>
        <button class="operator" onclick="setOperation('-')">-
</button>
        <button class="operator" onclick="setOperation('*')">*
</button>
        <button class="operator"
onclick="setOperation('/')">/</button>
        <br>
        <button style="width: 100%;" onclick="calculate()">=
</button>
    </div>

    <script src="script.js"></script>
</body>
</html>
```

script.js

```
let currentInput = '';
let previousInput = '';
let operation = null;

function appendNumber(number) {
        if (number === '.' && currentInput.includes('.')){
                return;
        }

        currentInput += number;
        updateDisplay();
}

function setOperation(op) {
        if (currentInput === ''){
                return;
        }

        previousInput = currentInput;
        operation = op;
        currentInput = '';
}

function clearDisplay() {
        currentInput = '';
        previousInput = '';
        operation = null;
        updateDisplay();
}

function calculate() {
        if (previousInput === '' || currentInput === ''){
                printToDisplay('Error');
                return;
        }
        let previous = parseFloat(previousInput);
        let current = parseFloat(currentInput);
        let result;

        switch(operation){
                case '+':
                        result = previous + current;
                        break;
                case '-':
```

```
                        result = previous - current;
                        break;
                case '*':
                        result = previous * current;
                        break;
                case '/':
                        if (current === 0){
                                printToDisplay('Cannot divide by
zero');
                                return;
                        }
                        result = previous / current;
                        break;
                default:
                        printToDisplay('Error');
                        return;
        }

        currentInput = result.toString();
        previousInput = '';
        operation = null;
        updateDisplay();
}

function updateDisplay() {
    document.getElementById('display').value = currentInput;
}

function printToDisplay(message) {
    document.getElementById('display').value = message;
}
```

S.No: 4	Exp. Name: ***Dynamic To-Do List with Modals***	**Date: 2026-09-21**

Aim:

You have joined a front-end team building a productivity app. The HTML and CSS for a To-Do List app are already provided. Write the JavaScript in `script.js` to make the app interactive.

Your Task

1. Add Task

When the user types a task and clicks the `Add` button, add a new `<li>` to the list with `id="task-list"`. Each item must include:

- A `<span>` holding the task text
- Two buttons labelled `Edit` and `Delete`

2. Edit Task with Modal

Clicking `Edit` on a task should:

- Open `#edit-modal` by setting its `display` to `block`
- Show the current task text inside `#edit-input`
- Update the task text when the user clicks `Save`

The modal should close when:

- The user clicks `Save`
- The user clicks the close icon at the top right
- The user clicks outside the modal

3. Delete Task with Confirmation

Clicking `Delete` on a task should:

- Open `#delete-modal` by setting its `display` to `block`
- Remove the task if the user clicks `Yes`
- Close the modal without deleting if the user clicks `No`
- Close the modal if the user clicks outside the confirmation box

Note:

- Trigger the **Add** action when the user clicks the **Add** button. You may implement this using either the button's $click$ event or the form's submit event (using `event.preventDefault()` if needed).

Source Code:

ID: A24126552275

ANIL NEERUKONDA INSTITUTE OF TECHNOLOGY & SCIENCES

index.html

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8" />
  <meta name="viewport" content="width=device-width, initial-
scale=1.0"/>
  <title>Dynamic To-Do List</title>
  <link rel="stylesheet" href="style.css" />
</head>
<body>
  <div class="container">
    <h1>To-Do List</h1>
    <form id="task-form">
      <input type="text" id="task-input" placeholder="Add a new
task" required />
      <button type="submit" id="add-task">Add</button>
    </form>
    <ul id="task-list"></ul>
  </div>

  <!-- Edit Modal -->
  <div id="edit-modal" class="modal">
    <div class="modal-content">
      <span class="close-btn" id="close-edit-modal">&times;
</span>
      <h2>Edit Task</h2>
      <input type="text" id="edit-input" />
      <button id="save-edit">Save</button>
    </div>
  </div>

  <!-- Delete Confirmation Modal -->
  <div id="delete-modal" class="modal">
    <div class="modal-content">
      <p>Are you sure you want to delete this task?</p>
      <button id="confirm-delete">Yes</button>
      <button id="cancel-delete">No</button>
    </div>
  </div>

  <script src="script.js"></script>
</body>
</html>
```

script.js

```
const taskForm = document.getElementById("task-form");
const tasklist = document.getElementById("task-list");
const taskInput = document.getElementById("task-input");

const editModal = document.getElementById("edit-modal");
const editInput = document.getElementById("edit-input");
const saveEdit = document.getElementById("save-edit");
const CloseEditModal = document.getElementById("close-edit-
modal");

const deleteModal = document.getElementById("delete-modal");
const confirmDelete = document.getElementById("confirm-delete");
const cancelDelete = document.getElementById("cancel-delete");
let currentTask=null;
taskForm.addEventListener("submit",function(event){
        event.preventDefault();
        const taskText=taskInput.value.trim();
        if (taskText===""){
                return;
        }
        const li=document.createElement("li");
        const span=document.createElement("span");
        span.textContent = taskText;
        const editButton=document.createElement("button");
        editButton.textContent = "Edit";
        const deleteButton=document.createElement("button");
        deleteButton.textContent = "Delete";
        li.appendChild(span);
        li.appendChild(editButton);
        li.appendChild(deleteButton);
        tasklist.appendChild(li);
        taskInput.value="";
});
tasklist.addEventListener("click",function(event){
        if (event.target.textContent ==="Edit"){
                currentTask=event.target.parentElement;

editInput.value=currentTask.querySelector("span").textContent;
                editModal.style.display = "block";
        }
        if (event.target.textContent === "Delete"){
                currentTask=event.target.parentElement;
                deleteModal.style.display = "block";
        }
```

```
});
saveEdit.addEventListener("click",function(){
        const newTask=editInput.value.trim();
        if (newTask !=="" && currentTask){

currentTask.querySelector("span").textContent=newTask;
                editModal.style.display = "none";
                currentTask=null;
        }
});
CloseEditModal.addEventListener("click",function(){
        editModal.style.display = "none";
        currentTask=null;
});
confirmDelete.addEventListener("click",function(){
        if(currentTask){
                currentTask.remove();
        }
        deleteModal.style.display = "none";
        currentTask=null;
});
cancelDelete.addEventListener("click",function(){
        deleteModal.style.display = "none";
        currentTask=null;
});
window.addEventListener("click",function(event){
        if (event.target === editModal){
                editModal.style.display = "none";
                currentTask=null;
        }
        if(event.target === deleteModal){
                deleteModal.style.display = "none";
                currentTask=null;
        }
});
```

S.No: 5	Exp. Name: ***Node.js Event Logger Using Packages, Events, Timers, and File I/O***	**Date: 2026-09-23**

Aim:

In this experiment, you will build a Node.js event logging system that connects four core concepts into one working flow. The `server.js`, browser UI, and CSS file are already provided in the default code. Your only task is to implement two things inside `logger.js`.

Your Task

Implement the following inside `logger.js`:

- Create a class **LogEmitter** that extends the built-in `EventEmitter` class.
- Implement the **startLogging** function. It must do the following:
    - Create an instance of **LogEmitter**.
- Register a listener on the `"log"` event. Each time the event fires, the listener must build a log entry in this exact format and append it to `LOG_FILE` using `fs.appendFileSync`:
- `[<uuid>] <timestamp> – <message>\n`
- where `uuid` comes from `uuidv4()` and `timestamp` comes from `new Date().toISOString()`.Use `setInterval` to emit the `"log"` event every `2000` ms with the message `"Event fired"`.Use `clearInterval` to stop the interval after 5 ticks.Emit a `"done"` event on the emitter after the 5th tick.Return the **LogEmitter** instance.

Note: Click **Start** in the browser UI to trigger your implementation and observe the live dashboard. Click **Reset** to clear and run again.

Source Code:

my-app/logger.js

```
const EventEmitter = require("events");
const fs = require("fs");
const path = require("path");
const { v4: uuidv4 } = require("uuid");

const LOG_FILE = path.join(__dirname, "logs", "events.log");

if (!fs.existsSync(path.join(__dirname, "logs"))) {
  fs.mkdirSync(path.join(__dirname, "logs"));
}

class LogEmitter extends EventEmitter{}
function startLogging() {
        const emitter = new LogEmitter();
                emitter.on("log",(message)=>{
                fs.appendFileSync(
                        LOG_FILE, `[${uuidv4()}] ${new
Date().toISOString()} - ${message}\n`
                );
        });
        let count = 0;
        const interval = setInterval(()=>{
                count++;
                emitter.emit("log", "Event fired");
                if (count === 5) {
                        clearInterval(interval);
                        emitter.emit("done");
                }
        }, 2000);
        return emitter;
}
module.exports = { startLogging, LOG_FILE };
```

my-app/package.json

```
{
  "name": "my-app",
  "version": "1.0.0",
  "description": "Node.js Events, Timers, Packages and File I/O",
  "main": "server.js",
  "scripts": {
    "start": "node server.js"
  },
  "dependencies": {
    "uuid": "^9.0.0"
  }
}
```

my-app/public/index.html

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8" />
  <meta name="viewport" content="width=device-width, initial-
scale=1.0"/>
  <title>Node.js Event Logger</title>
  <link rel="stylesheet" href="/style.css" />
</head>
<body>

  <div class="page-header">
    <h1>Node.js Event Logger</h1>
    <p class="subtitle">Monitor live event activity powered by
your Node.js implementation</p>
  </div>

  <div class="main">

    <div class="stat-grid">
      <div class="stat-card">
        <div class="stat-label">Status</div>
        <div class="stat-value" id="stat-status">
          <span class="badge idle">Idle</span>
        </div>
      </div>
      <div class="stat-card">
        <div class="stat-label">Ticks Fired</div>
        <div class="stat-value big" id="stat-ticks">0 <span
class="of">/ 5</span></div>
      </div>
      <div class="stat-card">
        <div class="stat-label">Last Event Time</div>
        <div class="stat-value mono" id="stat-time">—</div>
      </div>
      <div class="stat-card">
        <div class="stat-label">Log File Size</div>
        <div class="stat-value mono" id="stat-size">0 bytes</div>
      </div>
    </div>

    <div class="card">
      <div class="card-title">Last Generated UUID</div>
      <div class="uuid-display" id="stat-uuid">—</div>
```

```
    </div>

    <div class="card">
      <div class="card-title">Event Log</div>
      <div class="empty-state" id="empty-state">
        <span class="empty-icon">○</span>
        <p>No events yet. Click Start to begin.</p>
      </div>
      <div class="log-feed" id="log-feed"></div>
    </div>

    <div class="btn-row">
      <button id="start-btn"
onclick="handleStart()">Start</button>
      <button id="reset-btn" class="btn-secondary"
onclick="handleReset()">Reset</button>
    </div>

  </div>

  <script>
    let pollInterval = null;
    let lastCount = 0;

    function setStatus(type, label) {
      document.getElementById("stat-status").innerHTML = '<span
class="badge ' + type + '">' + label + '</span>';
    }

    window.handleStart = async function () {
      document.getElementById("start-btn").disabled = true;
      document.getElementById("reset-btn").disabled = true;
      setStatus("running", "Running");
      try {
        const res = await fetch("/start", { method: "POST" });
        const data = await res.json();
        if (data.status === "error") {
          setStatus("error", "Error");
          document.getElementById("stat-uuid").textContent =
data.message;
          document.getElementById("start-btn").disabled = false;
          document.getElementById("reset-btn").disabled = false;
          return;
        }
        document.getElementById("empty-state").style.display =
"none";
```

```
        lastCount = 0;
        pollInterval = setInterval(pollLogs, 600);
      } catch (err) {
        setStatus("error", "Error");
        document.getElementById("start-btn").disabled = false;
        document.getElementById("reset-btn").disabled = false;
      }
    };

    window.handleReset = async function () {
      clearInterval(pollInterval);
      await fetch("/reset", { method: "POST" });
      lastCount = 0;
      document.getElementById("stat-ticks").innerHTML = '0 <span
class="of">/ 5</span>';
      document.getElementById("stat-time").innerHTML = "—";
      document.getElementById("stat-size").textContent = "0
bytes";
      document.getElementById("stat-uuid").innerHTML = "—";
      document.getElementById("log-feed").innerHTML = "";
      document.getElementById("empty-state").style.display =
"flex";
      document.getElementById("start-btn").disabled = false;
      document.getElementById("reset-btn").disabled = false;
      setStatus("idle", "Idle");
    };

    async function pollLogs() {
      try {
        const res = await fetch("/logs");
        const data = await res.json();
        const lines = data.lines || [];

        if (lines.length !== lastCount) {
          lastCount = lines.length;
          document.getElementById("stat-ticks").innerHTML =
lines.length + ' <span class="of">/ 5</span>';
          document.getElementById("stat-size").textContent =
data.fileSize + " bytes";

          if (lines.length > 0) {
            const lastLine = lines[lines.length - 1];
            const uuidMatch = lastLine.match(/\[([^\]]+)\]/);
            const timeMatch = lastLine.match(/\]\s+([^\s]+)\s+/);
            if (uuidMatch) document.getElementById("stat-
uuid").textContent = uuidMatch[1];
```

```
            if (timeMatch) document.getElementById("stat-
time").textContent = timeMatch[1].replace("T", " ").replace("Z",
" UTC");
          }

          const feed = document.getElementById("log-feed");
          feed.innerHTML = lines.map(function(line, i) {
            const uuidMatch = line.match(/\[([^\]]+)\]/);
            const timeMatch = line.match(/\]\s+([^\s]+)\s+/);
            const msgMatch  = line.match(/\u2014\s+(.*)/);
            const uuid = uuidMatch ? uuidMatch[1] : "—";
            const time = timeMatch ? timeMatch[1].replace("T", "
").replace("Z", " UTC") : "—";
            const msg  = msgMatch  ? msgMatch[1] : line;
            return '<div class="log-row">'
              + '<span class="log-num">#' + (i + 1) + '</span>'
              + '<div class="log-details">'
              + '<span class="log-msg">' + msg + '</span>'
              + '<span class="log-meta">' + time + '</span>'
              + '<span class="log-uuid">' + uuid + '</span>'
              + '</div></div>';
          }).join("");
          feed.scrollTop = feed.scrollHeight;
        }

        if (!data.isLogging && lines.length >= 5) {
          clearInterval(pollInterval);
          setStatus("done", "Done");
          document.getElementById("start-btn").disabled = false;
          document.getElementById("reset-btn").disabled = false;
        }

        if (!data.isLogging && lines.length === 0) {
          clearInterval(pollInterval);
          setStatus("error", "Error");
          document.getElementById("start-btn").disabled = false;
          document.getElementById("reset-btn").disabled = false;
        }

      } catch (err) {
        clearInterval(pollInterval);
      }
    }
  </script>
```

```
</body>
</html>
```

my-app/public/style.css

```
* { box-sizing: border-box; margin: 0; padding: 0; }

body {
  font-family: "Segoe UI", sans-serif;
  background: #f0f2f5;
  min-height: 100vh;
  color: #1a202c;
}

.page-header {
  background: #fff;
  border-bottom: 1px solid #e2e8f0;
  padding: 24px 40px;
  text-align: center;
}

.page-header h1 {
  font-size: 1.4rem;
  font-weight: 700;
  color: #1a202c;
  margin-bottom: 4px;
}

.subtitle { font-size: 0.83rem; color: #718096; }

.main {
  max-width: 680px;
  margin: 32px auto;
  padding: 0 24px;
  display: flex;
  flex-direction: column;
  gap: 20px;
}

.stat-grid {
  display: grid;
  grid-template-columns: 1fr 1fr;
  gap: 14px;
}

.stat-card {
  background: #fff;
  border: 1px solid #e2e8f0;
  border-radius: 12px;
```

```
  padding: 18px 20px;
}

.stat-label {
  font-size: 0.72rem;
  font-weight: 700;
  text-transform: uppercase;
  letter-spacing: 0.07em;
  color: #a0aec0;
  margin-bottom: 10px;
}

.stat-value { font-size: 0.95rem; font-weight: 600; color:
#2d3748; }

.stat-value.big {
  font-size: 1.8rem;
  font-weight: 700;
  color: #1a202c;
  line-height: 1;
}

.stat-value.mono {
  font-family: monospace;
  font-size: 0.82rem;
  color: #4a5568;
  word-break: break-all;
}

.of { font-size: 1rem; color: #a0aec0; font-weight: 500; }

.badge {
  display: inline-block;
  padding: 3px 10px;
  border-radius: 20px;
  font-size: 0.75rem;
  font-weight: 700;
  text-transform: uppercase;
  letter-spacing: 0.05em;
}

.badge.idle    { background: #edf2f7; color: #718096; }
.badge.running { background: #ebf8ff; color: #2b6cb0; border: 1px
solid #90cdf4; }
.badge.done    { background: #f0fff4; color: #276749; border: 1px
solid #9ae6b4; }
```

```
.badge.error   { background: #fff5f5; color: #c53030; border: 1px
solid #feb2b2; }

.card {
  background: #fff;
  border-radius: 12px;
  border: 1px solid #e2e8f0;
  padding: 20px 24px;
}

.card-title {
  font-size: 0.72rem;
  font-weight: 700;
  text-transform: uppercase;
  letter-spacing: 0.07em;
  color: #a0aec0;
  margin-bottom: 12px;
}

.uuid-display {
  font-family: monospace;
  font-size: 0.88rem;
  color: #3182ce;
  font-weight: 600;
  background: #ebf8ff;
  border-radius: 7px;
  padding: 10px 14px;
  word-break: break-all;
  letter-spacing: 0.03em;
}

.empty-state {
  display: flex;
  flex-direction: column;
  align-items: center;
  justify-content: center;
  gap: 10px;
  padding: 32px 20px;
  text-align: center;
}

.empty-icon { font-size: 2rem; color: #e2e8f0; line-height: 1; }
.empty-state p { font-size: 0.83rem; color: #a0aec0; }

.log-feed {
  display: flex;
```

```
  flex-direction: column;
  gap: 8px;
  max-height: 280px;
  overflow-y: auto;
}

.log-row {
  display: flex;
  align-items: flex-start;
  gap: 12px;
  background: #f7fafc;
  border-radius: 8px;
  padding: 10px 14px;
  border-left: 3px solid #4299e1;
}

.log-num {
  font-size: 0.72rem;
  font-weight: 700;
  color: #a0aec0;
  min-width: 24px;
  padding-top: 2px;
}

.log-details {
  display: flex;
  flex-direction: column;
  gap: 3px;
  flex: 1;
}

.log-msg { font-size: 0.88rem; font-weight: 600; color: #2d3748;
}
.log-meta { font-size: 0.75rem; font-family: monospace; color:
#718096; }
.log-uuid { font-size: 0.72rem; font-family: monospace; color:
#a0aec0; word-break: break-all; }

.btn-row { display: flex; gap: 12px; }

button {
  flex: 1;
  padding: 11px;
  background: #3182ce;
  color: #fff;
  border: none;
```

```
  border-radius: 8px;
  font-size: 0.9rem;
  font-weight: 600;
  cursor: pointer;
  transition: background 0.2s, opacity 0.2s;
}

button:hover   { background: #2b6cb0; }
button:disabled { opacity: 0.5; cursor: not-allowed; }

.btn-secondary {
  background: #fff;
  color: #4a5568;
  border: 1.5px solid #e2e8f0;
}

.btn-secondary:hover { background: #f7fafc; }
```

my-app/server.js

```
const http = require("http");
const fs = require("fs");
const path = require("path");

const PORT = 3000;

const SERVER_FILE = path.join(__dirname, "logger.js");
let lastMtime = fs.statSync(SERVER_FILE).mtimeMs;
setInterval(() => {
  try {
    const mtime = fs.statSync(SERVER_FILE).mtimeMs;
    if (mtime !== lastMtime) {
      console.log("logger.js changed — restarting...");
      process.exit(1);
    }
  } catch (e) {}
}, 1000);

function serveStatic(res, filePath, contentType) {
  fs.readFile(filePath, (err, data) => {
    if (err) {
      res.writeHead(404, { "Content-Type": "text/plain" });
      res.end("404 Not Found");
      return;
    }
    res.writeHead(200, { "Content-Type": contentType });
    res.end(data);
  });
}

let isLogging = false;

function handleRequest(req, res) {
  if (req.method === "GET" && req.url === "/") {
    serveStatic(res, path.join(__dirname, "public",
"index.html"), "text/html");
    return;
  }

  if (req.method === "GET" && req.url === "/style.css") {
    serveStatic(res, path.join(__dirname, "public", "style.css"),
"text/css");
    return;
  }
```

```
  if (req.method === "POST" && req.url === "/start") {
    if (isLogging) {
      res.writeHead(200, { "Content-Type": "application/json" });
      res.end(JSON.stringify({ status: "already running" }));
      return;
    }
    try {
      delete require.cache[require.resolve("./logger")];
      const { startLogging, LOG_FILE } = require("./logger");
      if (typeof startLogging !== "function") {
        res.writeHead(500, { "Content-Type": "application/json"
});
        res.end(JSON.stringify({ status: "error", message:
"startLogging is not implemented yet." }));
        return;
      }
      const logDir = path.join(__dirname, "logs");
      if (!fs.existsSync(logDir)) fs.mkdirSync(logDir);
      fs.writeFileSync(LOG_FILE, "");
      isLogging = true;
      const emitter = startLogging();
      emitter.on("done", () => { isLogging = false; });
      res.writeHead(200, { "Content-Type": "application/json" });
      res.end(JSON.stringify({ status: "started" }));
    } catch (err) {
      isLogging = false;
      res.writeHead(500, { "Content-Type": "application/json" });
      res.end(JSON.stringify({ status: "error", message:
err.message }));
    }
    return;
  }

  if (req.method === "POST" && req.url === "/reset") {
    isLogging = false;
    try {
      delete require.cache[require.resolve("./logger")];
      const { LOG_FILE } = require("./logger");
      if (fs.existsSync(LOG_FILE)) fs.writeFileSync(LOG_FILE,
"");
    } catch (e) {}
    res.writeHead(200, { "Content-Type": "application/json" });
    res.end(JSON.stringify({ status: "reset" }));
    return;
  }
```

```
  if (req.method === "GET" && req.url === "/logs") {
    try {
      delete require.cache[require.resolve("./logger")];
      const { LOG_FILE } = require("./logger");
      const content = fs.existsSync(LOG_FILE) ?
fs.readFileSync(LOG_FILE, "utf8") : "";
      const lines = content.trim().split("\n").filter(Boolean);
      const fileSize = fs.existsSync(LOG_FILE) ?
fs.statSync(LOG_FILE).size : 0;
      res.writeHead(200, { "Content-Type": "application/json" });
      res.end(JSON.stringify({ lines, isLogging, fileSize }));
    } catch (err) {
      res.writeHead(200, { "Content-Type": "application/json" });
      res.end(JSON.stringify({ lines: [], isLogging: false,
fileSize: 0 }));
    }
    return;
  }

  res.writeHead(404, { "Content-Type": "text/plain" });
  res.end("Not Found");
}

const server = http.createServer(handleRequest);
server.listen(PORT, () => {
  console.log(`Server running at http://localhost:${PORT}`);
});
```

S.No: 6	Exp. Name: ***Developing a Single Page Application using RESTful APIs***	**Date: 2026-09-30**

ID: A24126552275

ANIL NEERUKONDA INSTITUTE OF TECHNOLOGY & SCIENCES

Aim:

You are developing a Movie Management System for an online streaming platform. The application allows users to add new movies, update movie ratings, delete movies, and view all available movies.

The application behaves as a Single Page Application (SPA). Instead of reloading the page after every operation, it communicates with the Express.js server using RESTful APIs through JavaScript's `fetch()` method.

The MongoDB connection, Movie model, project configuration, HTML, CSS, and the basic JavaScript functions have already been provided. Your task is to complete the missing code in both **index.js** and **script.js** so that the application works correctly.

Your Tasks

Complete the missing code in **index.js** by implementing the following RESTful API routes.

1. Create Movie

Create a POST route at `/add-movie` that:

- Accepts book details ($title$, $director$, $year$, and $rating$) in JSON format.
- Stores the received data as a new **Movie** document in the database.

Responds with:

```
Movie added: <movie-title>
```

2.View Movies

Create a GET route at `/movies` that:

- Retrieves all movies from the database using `Movie.find()`.
- Returns the retrieved documents as JSON.
- If an error occurs, respond with status code 500 and an appropriate error message.

3. Update Movie rating

Create a PUT route at `/update-movie` that:

- Accepts $title$ and $newRating$ from the request body.
- Updates only the rating of the matching movie using the $set operator.
- If the update is successful, respond with:

```
Movie '<title>' rating updated successfully
```

- If no matching movie is found, respond with:

```
No movie found with title '<title>'
```

- If an error occurs, respond with status code `500`.

4. Delete Movie

Create a DELETE route at `/movies/:title` that:

- Deletes the movie whose title matches the route parameter.

- If the movie is deleted successfully, respond with: {"message": "Movie deleted successfully"}
- If no movie exists respond with: {"message": "Movie not found"}
- If an error occurs, respond with status code `500` and an appropriate error message.

Complete the missing code in **script.js** by completing `fetch()` requests for the following functions.

- **addMovie()** → Write a `fetch()` request that sends a `POST` request to `/add-movie` with the provided movie details in JSON format.
- **updateMovie()** → Write a `fetch()` request that sends a `PUT` request to `/update-movie` with the provided title and newRating in JSON format.
- **deleteMovie()** → Write a `fetch()` request that sends a `DELETE` request to `/movies/:title` by appending the movie title from the input field, encoded using `encodeURIComponent()`, to `/movies/`.
- **viewMovies()** → Write a `fetch()` request that sends a `GET` request to `/movies` and retrieves the movie list for display.

Run the application and verify that movies can be added, deleted, updated, and retrieved successfully.

Note: Do not modify the provided application setup. Write only the missing code in **index.js** and **script.js**.

Source Code:

ID: A24126552275

ANIL NEERUKONDA INSTITUTE OF TECHNOLOGY & SCIENCES

express-app/index.js

```
const express = require('express');
const mongoose = require('mongoose');
const path = require('path');
const Movie = require('./models/movieModel');

const app = express();
app.use(express.json());
app.set('view engine', 'ejs');
app.use(express.urlencoded({ extended: true }));
app.use(express.static(path.join(__dirname, 'public')));

let dbName = '';
let dbHost = '';
let dbPort = '';

mongoose.connect('mongodb://mongo:27017/libraryDB', {
  useNewUrlParser: true,
  useUnifiedTopology: true
})
.then(() => {
  console.log('Database connected successfully!');

  const db = mongoose.connection;
  dbName = db.name;
  dbHost = db.host;
  dbPort = db.port;
})
.catch((err) => {
  console.error('Database connection failed:', err.message);
});

app.get('/ui', (req, res) => {
  res.render('index');
});

// Write POST /add-movie
app.post('/add-movie',async(req,res)=>{
        try {
                const {title, director,year,rating}= req.body;
                const movie = new
Movie({title,director,year,rating});
                await movie.save();
                res.send(`Movie added: ${movie.title}`);
        }
```

```
        catch (err) {
                res.status(500).send('Error adding movie:
'+err.message);
        }
});


// Write GET /movies
app.get('/movies',async(req,res)=>{
        try {
                const movies = await Movie.find();
                res.json(movies);
        }
        catch (err){
                res.status(500).send('Error retrieving movies:
'+err.message);
        }
});

// Write PUT /update-movie
app.put('/update-movie',async (req,res)=>{
        try{
                const {title, newRating }=req.body;
                const updated=await Movie.findOneAndUpdate(
                        { title },
                        { $set: {rating: newRating }},
                        { new: true }
                );
                if (!updated) {
                        return res.send(`No movie found with
title '${title}'`);
                }
                res.send(`Movie '${title}' rating updated
successfully`);
        }
        catch (err) {
                res.status(500).send('Error updating movie:
'+err.message);
        }
});

// Write DELETE /movies/:title
app.delete('/movies/:title', async (req,res)=>{
        try {
                const deleted = await Movie.findOneAndDelete({
title: req.params.title });
```

```
                if (!deleted) {
                        return res.json({ message: "Movie not
found" });
                }
                res.json({ message: "Movie deleted successfully"
});
        }
        catch (err) {
                res.status(500).json({ message: 'Error deleting
movie: '+err.message });
        }
});

// -------------------- DATABASE DETAILS --------------------

app.get('/', (req, res) => {
  res.json({
    dbName,
    dbHost,
    dbPort
  });
});


// -------------------- SERVER --------------------

app.listen(3000, () => {
  console.log('Server running on http://localhost:3000');
});
```

express-app/public/script.js

```
function hideAll() {
    document.querySelectorAll(".page").forEach(page =>
page.style.display = "none");
}

function showPage(id) {
    hideAll();
    document.getElementById(id).style.display = "block";
    document.getElementById("output").innerHTML = "";
}

function goHome() {
    hideAll();
    document.getElementById("home").style.display = "block";
    document.getElementById("output").innerHTML = "";
}


async function addMovie() {
        const response = await fetch('/add-movie', {
                method: 'POST',
                headers: { 'Content-Type': 'application/json' },
        body: JSON.stringify({
            title: document.getElementById("title").value,
            director: document.getElementById("director").value,
            year: Number(document.getElementById("year").value),
            rating:
Number(document.getElementById("rating").value)
        })
        });
    document.getElementById("output").innerHTML = await
response.text();
}

async function updateMovie() {
        const response=await fetch('/update-movie', {
                method: 'PUT',
                headers: { 'Content-Type': 'application/json' },
        body: JSON.stringify({
          title: document.getElementById("updateTitle").value,
          newRating:
Number(document.getElementById("newRating").value)
      })
        });
```

```
    document.getElementById("output").innerHTML = await
response.text();
}

// Write your code here for Delete Movie

async function deleteMovie() {
        const title=document.getElementById("deleteTitle").value;
        const response=await fetch('/movies/'
+encodeURIComponent(title), {
                method: 'DELETE'
        });
        const data=await response.json();
    document.getElementById("output").innerHTML = await
response.text();
}

async function viewMovies() {
    hideAll();
    document.getElementById("viewPage").style.display = "block";
// write the fetch to view
        const response=await fetch('/movies');
        const movies=await response.json();
    let table = `
    <table>
        <tr>
            <th>Title</th>
            <th>Director</th>
            <th>Year</th>
            <th>Rating</th>
        </tr>
    `;

    movies.forEach(movie => {

        table += `
        <tr>
            <td>${movie.title}</td>
            <td>${movie.director}</td>
            <td>${movie.year}</td>
            <td>${movie.rating}</td>
        </tr>
        `;

    });
```

```
    table += "</table>";
    document.getElementById("movieTable").innerHTML = table;
}
```

S.No: 7	Exp. Name: ***Creating and Nesting Function and Class Components***	**Date: 2026-09-30**

Aim:

You are developing a React application for a digital library. The application should display information about a book by creating and nesting React components. In this activity, the React application setup has already been provided. Your task is to create the **Book** function component and the **App** class component and nest the **Book** component inside the **App** component.

Your Task

1. Inside **Book.jsx**, create a function component named **Book** that returns a `<div>` with the class **book-card**, containing:
   - An `<h2>` element with the text "The Great Gatsby".
   - A paragraph element with the text "Author: F. Scott Fitzgerald".

Export the **Book** component.

2. In the **App.jsx** file:
   - Create a class component named **App** that extends **Component**.
   - Import the `Book` component from `./Book`.
   - Inside the `render()` method, display an `<h3>` element with the text "Books".
   - Create a `<div>` with the class name **library**, and nest the `<Book />` component inside it.
   - Return the JSX from the `render()` method.
   - Export the **App** component as the default export.

Expected Output:

When the application is executed, it should display:

```
Books
--------------------------------
The Great Gatsby
Author: F. Scott Fitzgerald
--------------------------------
```

where the **Book** component is nested inside the **App** component.

Note:

- Both **App.jsx** and **Book.jsx** are in the same path.
- Initially, the page will show a placeholder until the required code is completed.

Source Code:

functionAndClassComponents/src/App.jsx

```
import { Component } from "react";
// Import the Book component here.
import Book from "./Book";
import ErrorBoundary from "./components/ErrorBoundary";

// Create the App class component here...
class App extends Component{
        render(){
                return (
                        <div>
                                <h3>Books</h3>
                                <div className="library">
                                        <Book />
                                </div>
                        </div>
                );
        }
}
export default App;
```

functionAndClassComponents/src/Book.jsx

```
// Create the Book function component here...
function Book(){
        return(
                <div className="book-card">
                        <h2>The Great Gatsby</h2>
                        <p>Author: F. Scott Fitzgerald</p>
                </div>
        );
}
export default Book;
```